What Impacts City of Calgary Property Values?

Modelling property assessment values using regression, KNN, random forests, gradient boosting

DATA 607 Group Members:

- Javier Becaria UCID: 30234458
- Chris Petrauskas UCID: 30270696

Dataset

CITY OF CALGARY HISTORICAL PROPERTY ASSESSMENT VALUES

*Dataset sourced from the **City of Calgary Open Data Portal**, containing parcel-level historical property assessments since 2005. Covers residential, non-residential, and farmland properties; freely available with attribution.*

Feature	Description	Type
ROLL_YEAR	Assessment year	Numerical (time)
ADDRESS	Street address	Categorical (text)
ASSESSED_VALUE	Total property value	Numerical
ASSESSMENT_CLASS	Tax classification code	Categorical
ASSESSMENT_CLASS_DESC	Tax class description	Categorical (text)
RE_ASSESSED_VALUE	Residential value	Numerical
NR_ASSESSED_VALUE	Non-residential value	Numerical
FL_ASSESSED_VALUE	Farmland value	Numerical
COMM_CODE	Community code	Categorical
COMM_NAME	Community name	Categorical (text)
YEAR_OF_CONSTRUCTION	Year built	Numerical (discrete)
LAND_USE_DESIGNATION	Zoning category	Categorical
PROPERTY_TYPE	Property type	Categorical
LAND_SIZE_SM	Land size (m^2)	Numerical
LAND_SIZE_SF	Land size (ft^2)	Numerical
LAND_SIZE_AC	Land size (acres)	Numerical
SUB_PROPERTY_USE	Detailed usage	Categorical
MULTIPOLYGON	Parcel geometry	Geospatial
CPID	Property ID	Categorical (ID)
VALUE_PER_SM	Value per m^2	Numerical

Target Variable ➡

Data Preparation

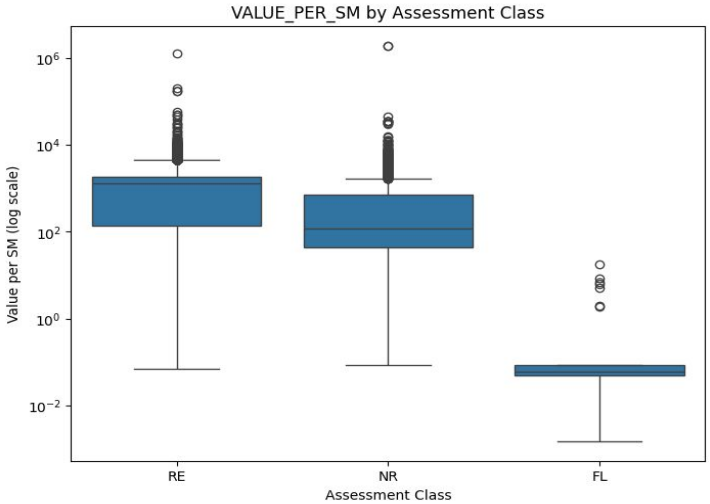
Sampling and Cleaning

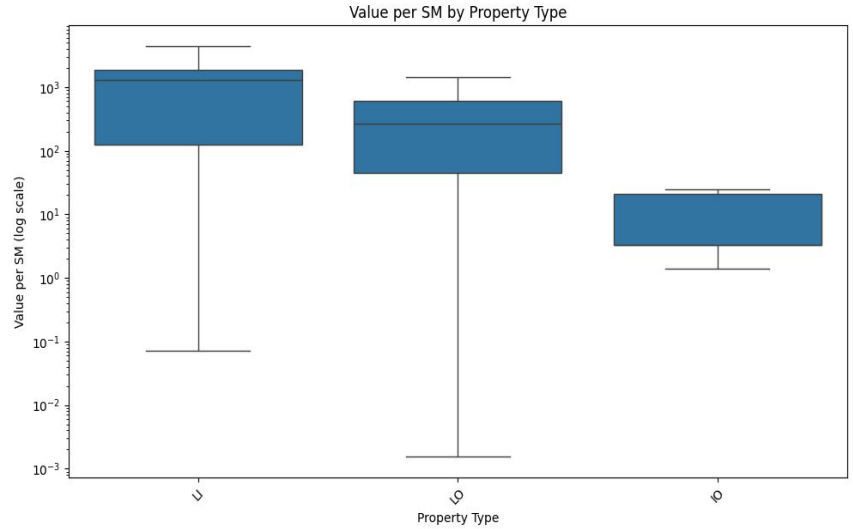
- Very large dataset (~10M rows, ~15 GB)
  - Read data in chunks
  - Formatted data types
  - Filtered to last 7 roll years
- Sampling strategy
  - Sampled 20% of unique roll numbers
  - Preserved full history per property (not random rows)
- Storage optimization
  - Converted to parquet format
- Data cleaning
  - Removed empty / corrupted rows
  - Filled missing values
  - Static fields (e.g., year built, land size) → earliest non-null
  - Dynamic fields (e.g., land use) → most recent or group median

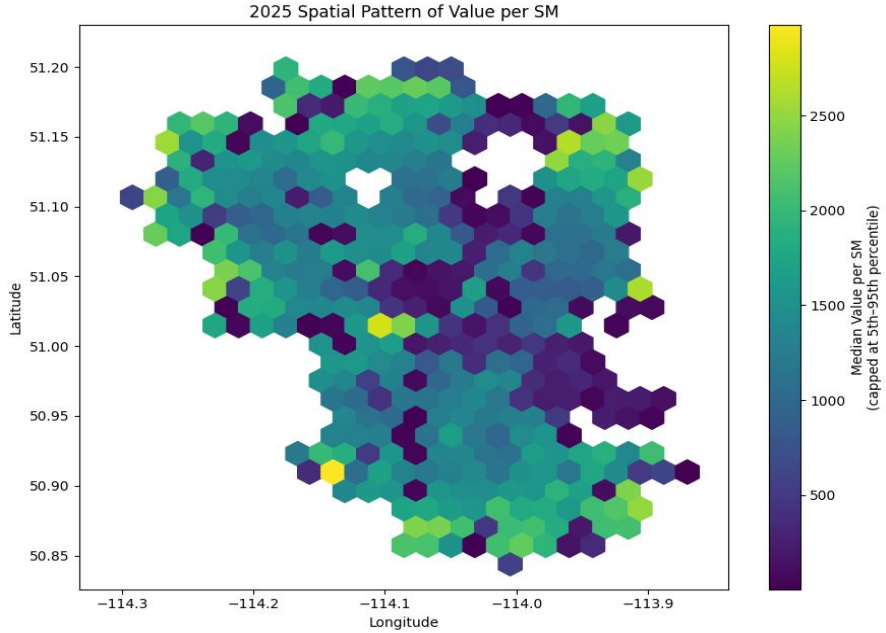
Feature Engineering and Split

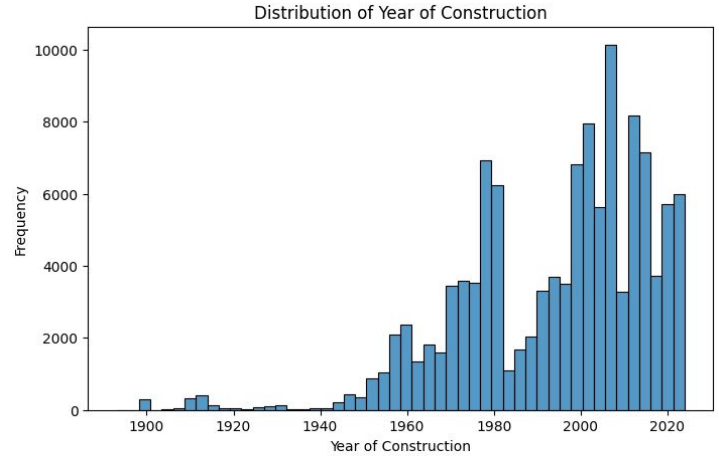
- Feature engineering
  - Value per m^2 (target)
  - Lat/long from polygon
  - Distance from Calgary centreCity quadrants (NW, NE, SW, SE, CENTRAL)
  - Density proxy (properties per community)
  - Value of 10 nearest properties
- Final dataset
  - 118k rows
  - Size reduced: 358 MB → 41 MB (parquet)
- Analysis approach
  - Started with 2025 cross-section
  - Then expanded to historical data
- Train, Val, Test = 90:10:10

Exploratory Data Analysis

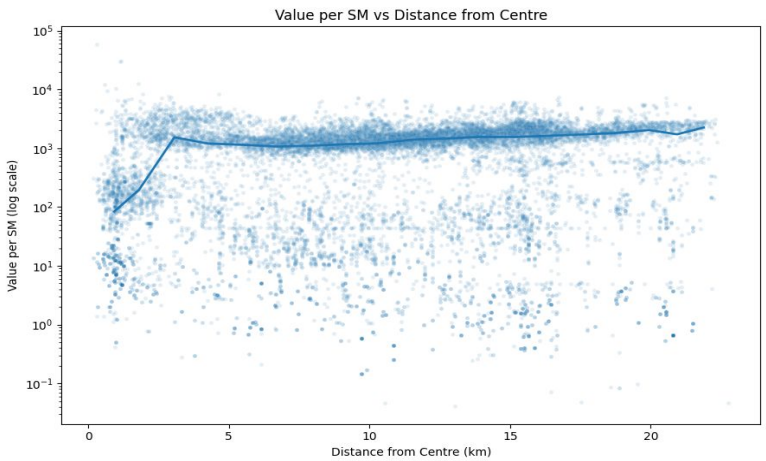


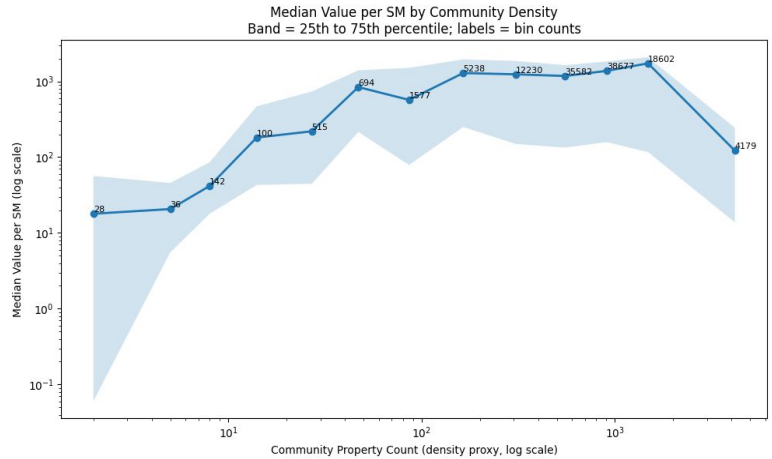


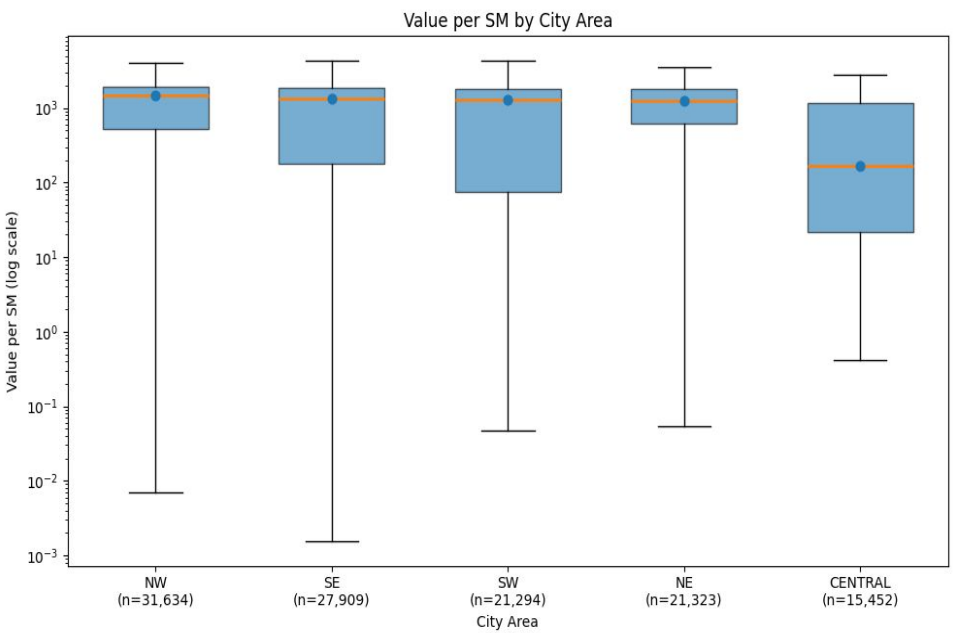


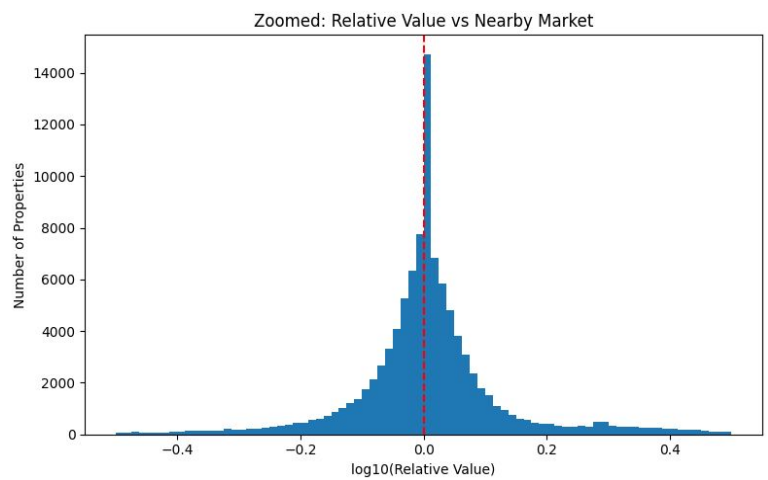


EDA


Value per SM vs Distance from Centre
Value per SM (log scale)
Distance from Centre (km)


Median Value per SM by Community Density
Band = 25th to 75th percentile; labels = bin counts
Median Value per SM (log scale)
Community Property Count (density proxy, log scale)
28
36
142
100
515
694
1577
5238
12230
35582
38677
18602
4179


Value per SM by City Area
Value per SM (log scale)
NW
(n=31,634)
SE
(n=27,909)
SW
(n=21,294)
NE
(n=21,323)
CENTRAL
(n=15,452)
City Area


Zoomed: Relative Value vs Nearby Market
Number of Properties
log10(Relative Value)

Ridge and Lasso Regression Models (2025 Only)

```python
# ----------------------------
# 3) Preprocessing
# ----------------------------
numeric_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer([
    ("num", numeric_transformer, numeric_features),
    ("cat", categorical_transformer, categorical_features)
])

# ----------------------------
# 4) Alpha grid
# ----------------------------
alphas = np.logspace(-4, 1, 20)

# ----------------------------
# 5) Ridge CV (L2)
# ----------------------------
ridge_cv_model = Pipeline([
    ("preprocessor", preprocessor),
    ("model", RidgeCV(alphas=alphas, cv=5))
])

ridge_cv_model.fit(X_train, y_train)

y_train_pred_ridge = ridge_cv_model.predict(X_train)
y_val_pred_ridge = ridge_cv_model.predict(X_val)

print("RIDGE CV (L2)")
print("Best alpha:", ridge_cv_model.named_steps["model"].alpha_)
print("Train R-squared:", r2_score(y_train, y_train_pred_ridge))
print("Validation R-squared:", r2_score(y_val, y_val_pred_ridge))
print("Validation RMSE:", np.sqrt(mean_squared_error(y_val, y_val_pred_ridge)))
print()

# ----------------------------
# 6) Lasso CV (L1)
# ----------------------------
lasso_cv_model = Pipeline([
    ("preprocessor", preprocessor),
    ("model", LassoCV(
        alphas=alphas,
        cv=5,
        max_iter=20000,
```

Model Results

Target var = log value_per_SM

Model	Best Alpha	Train R^2	Validation R^2	Validation RMSE
Ridge (L2)	0.079	0.742	0.720	1.193
Lasso (L1)	0.000	0.742	0.720	1.193

! BUT … without nearby median value feature RMSE doubled…

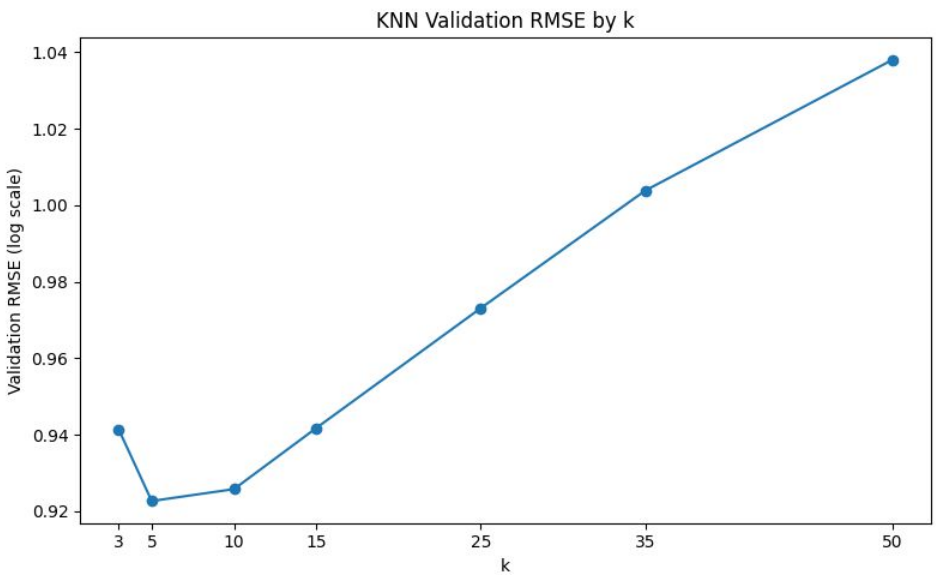
! Excluded lot size as realized dependent relationship

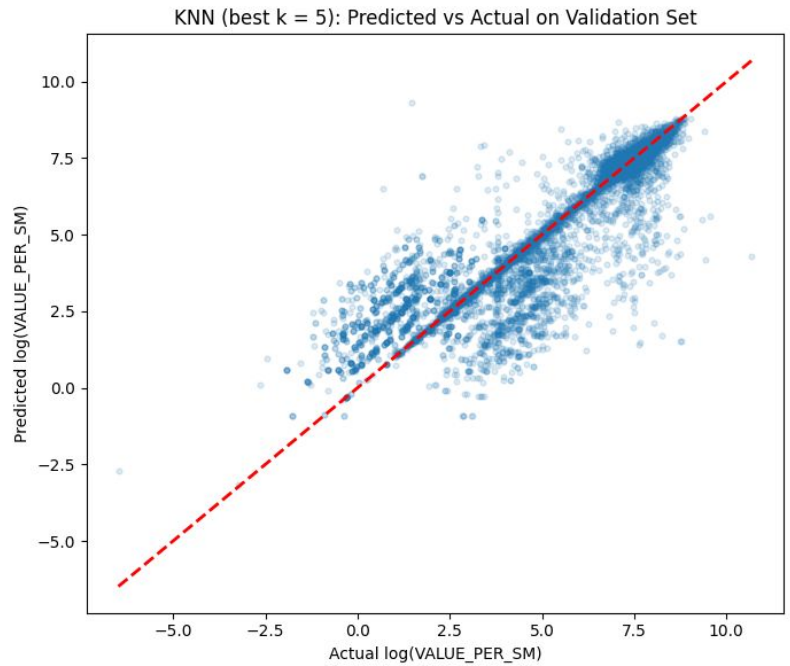
```
RIDGE COEFFICIENTS (ordered by importance)
                              Feature  Coefficient
4                ASSESSMENT_CLASS_FL       -4.483
6                ASSESSMENT_CLASS_RE        2.477
7                   PROPERTY_TYPE_IO       -2.224
5                ASSESSMENT_CLASS_NR        2.006
3   LOG_NEARBY_MEDIAN_VALUE_PER_SM        1.926
8                   PROPERTY_TYPE_LI        1.809
9                   PROPERTY_TYPE_LO        0.415
0                    AGE_OF_PROPERTY        0.046
10                  CITY_AREA_CENTRAL        0.040
2            LOG_COMM_PROPERTY_COUNT       -0.038
12                      CITY_AREA_NW       -0.028
1                DIST_FROM_CENTRE_KM        0.024
11                      CITY_AREA_NE       -0.010
14                      CITY_AREA_SW       -0.003
13                      CITY_AREA_SE        0.001

LASSO COEFFICIENTS (ordered by importance)
                              Feature  Coefficient
4                ASSESSMENT_CLASS_FL       -6.316
3   LOG_NEARBY_MEDIAN_VALUE_PER_SM        1.926
7                   PROPERTY_TYPE_IO       -1.865
8                   PROPERTY_TYPE_LI        1.396
6                ASSESSMENT_CLASS_RE        0.472
0                    AGE_OF_PROPERTY        0.045
10                  CITY_AREA_CENTRAL        0.040
2            LOG_COMM_PROPERTY_COUNT       -0.037
12                      CITY_AREA_NW       -0.025
1                DIST_FROM_CENTRE_KM        0.023
11                      CITY_AREA_NE       -0.007
13                      CITY_AREA_SE        0.004
5                ASSESSMENT_CLASS_NR        0.000
9                   PROPERTY_TYPE_LO       -0.000
14                      CITY_AREA_SW        0.000
```

KNN Model

```python
# ----------------------------
# 3) Preprocessing
# ----------------------------
numeric_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer([
    ("num", numeric_transformer, numeric_features),
    ("cat", categorical_transformer, categorical_features)
])

# ----------------------------
# 4) Try different k values
# ----------------------------
k_values = [3, 5, 10, 15, 25, 35, 50]
results = []
fitted_models = {}

for k in k_values:
    knn_model = Pipeline([
        ("preprocessor", preprocessor),
        ("model", KNeighborsRegressor(n_neighbors=k, weights="distance"))
    ])

    knn_model.fit(X_train, y_train)

    y_train_pred = knn_model.predict(X_train)
    y_val_pred = knn_model.predict(X_val)

    results.append({
        "k": k,
        "Train R-squared": r2_score(y_train, y_train_pred),
        "Validation R-squared": r2_score(y_val, y_val_pred),
        "Train RMSE (log scale)": np.sqrt(mean_squared_error(y_train, y_train_pred)),
        "Validation RMSE (log scale)": np.sqrt(mean_squared_error(y_val, y_val_pred)),
        "Validation RMSE (original scale)": np.sqrt(
            mean_squared_error(np.exp(y_val), np.exp(y_val_pred))
        )
    })

    fitted_models[k] = knn_model

results_df = pd.DataFrame(results)
```

Model Results





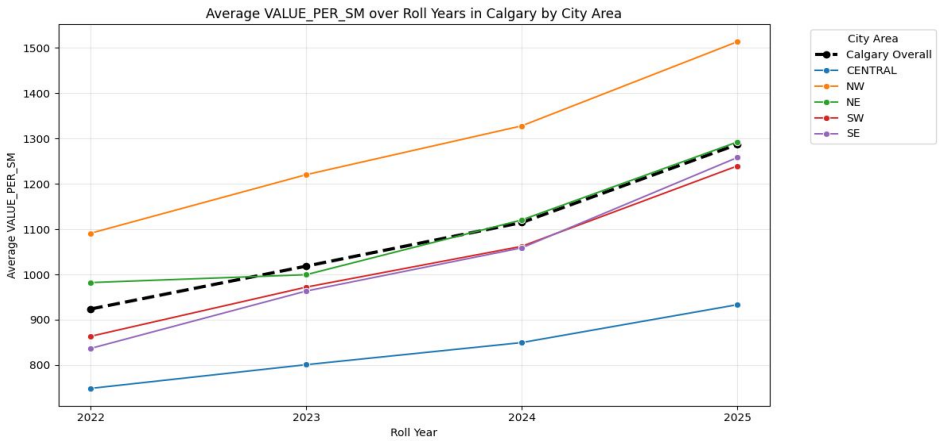
Model	Best k	Train R^2	Validation R^2	Train RMSE (log)	Validation RMSE (log)	Train RMSE	Validation RMSE
KNN Regressor	5	0.887	0.832	0.764	0.923	122.1	673.2

BEST KNN FEATURE IMPORTANCE

	Feature	Importance	Std
1	center_latitude	1.185	0.009
2	center_longitude	1.183	0.012
0	AGE_OF_PROPERTY	1.107	0.013
3	ASSESSMENT_CLASS	0.079	0.004
4	PROPERTY_TYPE	0.060	0.002

TIME BASED EVALUATION

Target: Property Value CAN$/m2 (Numerical)



Ascending trend year over year since 2021 with a higher growth between 2024-2025

DATA SPLIT

Train: 2021-2023	Val: 2024	Test: 2025

Create train, val, test datasets

```
# -------------------------
# 1) Split
# -------------------------
df_train = df[df['ROLL_YEAR'] <= 2023].copy()
df_val   = df[df['ROLL_YEAR'] == 2024].copy()
df_test  = df[df['ROLL_YEAR'] == 2025].copy()

# -------------------------
# 2) Sample 20%
# -------------------------
df_train = df_train.sample(frac=0.20, random_state=42)
df_val   = df_val.sample(frac=0.20, random_state=42)
df_test  = df_test.sample(frac=0.20, random_state=42)
```

MODELS

RF

```python
# ----------------------------
# 3) Preprocessing
# ----------------------------
numeric_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer([
    ("num", numeric_transformer, numeric_features),
    ("cat", categorical_transformer, categorical_features)
])

from sklearn.pipeline import Pipeline
from sklearn.metrics import r2_score, mean_squared_error

# ----------------------------
# 4) Random Forest pipeline
# ----------------------------
rf_pipeline = Pipeline([
    ("preprocessor", preprocessor),
    ("rf", RandomForestRegressor(
        n_estimators=500,
        max_depth=25,
        min_samples_split=75,
        min_samples_leaf=35,
        max_features="sqrt",
        random_state=42,
        n_jobs=-1
    ))
])

# ----------------------------
# 5) Fit
# ----------------------------
rf_pipeline.fit(X_train, y_train)

# ----------------------------
# 6) Predict and evaluate
# ----------------------------
y_train_pred = rf_pipeline.predict(X_train)
y_val_pred   = rf_pipeline.predict(X_val)
```

GB

```python
# ----------------------------
# 1) Apply preprocessing
# ----------------------------
X_train_processed = preprocessor.fit_transform(X_train)
X_val_processed   = preprocessor.transform(X_val)

# ----------------------------
# 2) Gradient Boosting configuration
# ----------------------------
staged_cfg = dict(
    learning_rate=0.1,
    n_estimators=2000,
    max_depth=4
)
gb_staged = GradientBoostingRegressor(random_state=607, **staged_cfg)

# ----------------------------
# 3) Fit the model on processed training data
# ----------------------------
gb_staged.fit(X_train_processed, y_train)

# ----------------------------
# 4) Track RMSE for each iteration
# ----------------------------
train_rmse_path = []
val_rmse_path = []

for y_tr_pred, y_val_pred in zip(
    gb_staged.staged_predict(X_train_processed),
    gb_staged.staged_predict(X_val_processed)
):
    train_rmse_path.append(np.sqrt(mean_squared_error(y_train, y_tr_pred)))
    val_rmse_path.append(np.sqrt(mean_squared_error(y_val, y_val_pred)))
```

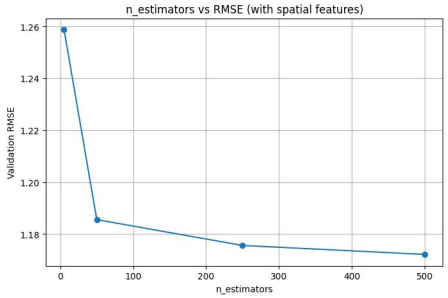
- Code extracted from Data 607 - Lecture 4. Dr. Ramazi.

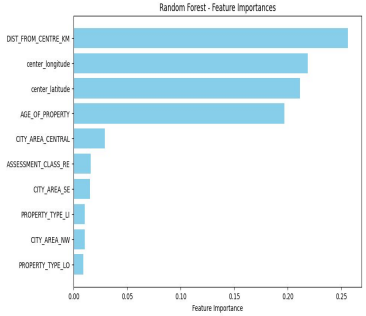
RF/GB

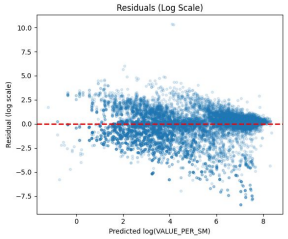
```
# ----------------------------
# 2) Numeric / categorical columns
# ----------------------------
numeric_features = [
    "ROLL_YEAR","AGE_OF_PROPERTY",
    "DIST_FROM_CENTRE_KM",
    "center_latitude",
    "center_longitude"
]

categorical_features = [
    "ASSESSMENT_CLASS",
    "PROPERTY_TYPE",
    "CITY_AREA"
]
```

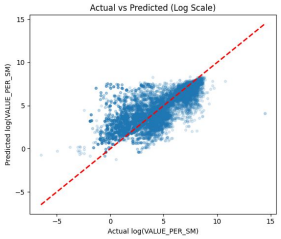
RF

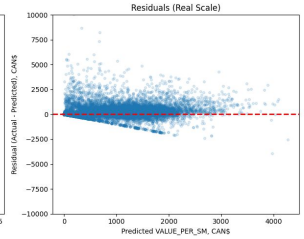
- Tuned n_ of_ trees ~500
- Difficult to tune depth, split and leaf number due to run time constrain.
- Main Features: LAT, LONG, DISTANCE TO CITY CENTER
- Reasonable prediction, high Log Error, high error on actual variable.





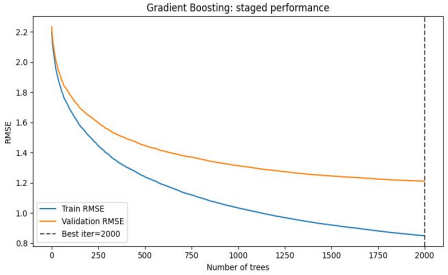


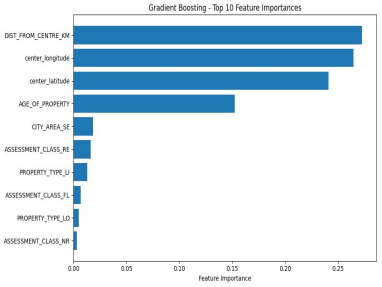


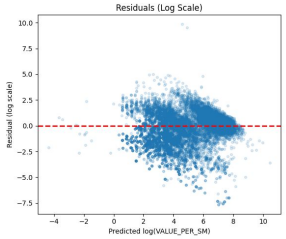


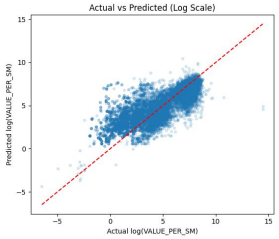
GB

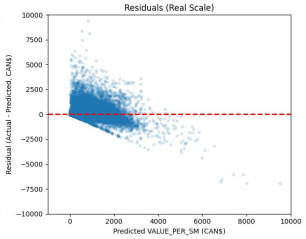
- Tuned n_of_trees ~2000
- Allowed tuning on
  Learning rate = 0.1
  Max depth = 4
- Main Features: LAT, LONG, DISTANCE TO CITY CENTER
- Reasonable prediction, high Log Error, high error on actual variable.











Model Results

Model	Train R^2	Train RMSE	Validation R^2	Validation RMSE
OLS_RidgeCV	0.6012	1.4063	0.5565	1.5023
Random Forest	0.5422	1.5067	0.4638	1.6519
Gradient Boost	0.8548	0.8487	0.7122	1.2102

Model	Advantages	Limitations
OSL	- Simple and easy to implement on large datasets - Provides early insights into trends of value per m² in Calgary - Work better when **Ridge** was used for parameter selection.	- Did not capture high variations, even in the 2024 validation data that appeared similar to previous years
Random Forest	- Requires early tuning to achieve reasonable performance - Captures non-linearity, but the model remained underfitted	- Increasing number of trees, depth, and minimum leaf size slows down model training - No clear optimization observed - slow running time constraints - Requires additional tuning (current r2<0.5) - Shows lower performance compared to OLS
Gradien Boosting	- Number of trees, learning rate, and max depth were successfully tuned to minimize error - Faster and more suitable for large datasets - Captures non-linear patterns in the target variable - Achieved reasonable r2 values on both training and validation datasets	- Be careful not to overfit when increasing the learning rate or the number of trees - Performance may deteriorate if the model becomes too complex

Conclusions

SINGLE YEAR EVALUATION:

1. **A single feature (nearby market value) drives most predictive power.** Removing it causes a major drop in performance → most of the signal is local.
2. **KNN outperforms regression in predictive accuracy.** Using actual nearby observations beats trying to approximate the relationship linearly.
3. **Regression is useful for interpretation, not performance.** It clearly shows what matters, but not necessarily the best way to predict.
4. **Property-specific features have limited incremental value.** Once local market context is included, features like property type or age add relatively little.
5. **The problem is fundamentally spatial, not structural.** Value is better understood as a function of where a property is rather than what it is.

MULTIPLE YEARS EVALUATION:

1. Splitting the data by year for train/validation did not reduce the observed error, whether done within the same year or ignoring the year split
2. Adding `ROLL_YEAR` as a feature did not improve model performance
3. Most significant features across models:
   - **Time**: Property age
   - **Location**: Latitude, longitude, distance from city center

   Used in spatiotemporal and hedonic models for property price prediction
4. Gradient Boosting and its variations (e.g., XGBoost) could improve accuracy, potentially by incorporating local economic indicators such as Bank of Canada interest rates, GDP, etc.

Future Improvements

1. **Model optimization**: Further hyperparameter tuning could reduce current errors
2. **Features**: Including local economic indicators and census data may improve predictions - features in this dataset are limited
3. **Evaluation & modeling**: Time series techniques—such as lag features and rolling averages—could better capture year-over-year property price variations

Thank you!
Any Questions?